

1) Discuss the tic-tac-toe problem in detail and explain how it can be solved using AI technique.

Answer: Tic-Tac-Toe

Tic-tac-toe (also known as **noughts and crosses** or **Xs and Os**) is a paper-and-pencil game for two players, X and O, who take turns marking the spaces in a 3×3 grid. The player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal row wins the game. The following example game is won by the first player, X:



Players soon discover that the best play from both parties leads to a draw. Hence, tic-tac-toe is most often played by young children.

Because of the simplicity of tic-tac-toe, it is often used as a pedagogical tool for teaching the concepts of good sportsmanship and the branch of artificial intelligence that deals with the searching of game trees. It is straightforward to write a computer program to play tic-tac-toe perfectly or to enumerate the 765 essentially different positions (the state space complexity) or the 26,830 possible games up to rotations and reflections (the game tree complexity) on this space.

The programs in this series increase in:

- Their complexity
- Their use of generalizations
- The clarity of knowledge
- The extensibility of their approach. Thus, they move toward being representations of what we call AI techniques.

PROGRAM 1

Data Structure:

Board: A nine-element vector representing the board, where the elements of the vector correspond to the board position as follows:

	1	2	3	
	4	5	6	
	7	8	9	

An element contains the value 0 if the corresponding square is blank, 1 if it is filled with an X, or 2 if it is filled with an O.

Move table: A large vector of 19683 elements (3^9), each element of which is a nine-element vector.

Algorithm: To make a move, do the following:

1. View the vector board as a ternary number (base three). Convert it to a decimal number.
2. Use the number computed in step 1 as index into Move table and access the vector stored there.
3. The vector selected in step 2 represents the way the board will look after the move that should be made. So set Board equal to that vector.

Advantages:

- a. This program is very efficient in terms of time,
- b. optimal game of tic-tac-toe in theory

Disadvantages:

- c. It takes a lot of space to store the table that specifies the correct move to make from each board position.
- d. Someone will have to do a lot of work specifying all the entries in the

movetable.

- e. It is very unlikely that all the required Movetable entries can be determined and entered without any errors.
- f. If we want to extend the game, say to three dimensions - 3^{27} , no longer work at all.

PROGRAM 2

Data Structure:

Board: A nine-element vector representing the board, where the elements of the vector correspond to the board position as follows:

1	2	3
4	5	6
7	8	9

But instead of using the numbers, 0, 1, or 2 in each element, we store 2 (indicating blank), 3 (indicating X) or 5 (indicating O).

Turn: An integer indicating which move of the game is about to be played: 1 indicates the first move, 9 the last.

Algorithm: The main algorithm uses 3 *sub procedures*

Make2: Returns 5 if the center square of the Board [5] is blank, that is if Board [5] = 2. Otherwise return any blank non-corner square [2, 4, 6, or 8],

Posswin(p): Returns 0 if player p cannot win on his next move; otherwise it returns the number of the square that constitutes a winning move.

If product is 18 ($3*3*2$), then X can win.

If the product is 50 ($5*5*2$), then O can win.

- **Go (n):** Move to Board [n] to 3 if Turn is odd, or 5 if Turn is even. It also increments Turn by one.
- Turn=1 Go (1)
- Turn=2 Go (5) or Go (1)
- Turn=3 Go (9) or Go (3)
- Turn=4 Go(Posswin(X)) or Go(Make2)
- Turn=5 Go(Posswin(X)), or Go(Posswin(O)), or Go(7), or Go(3) [fork]
- ...

Comments

- Less efficient than Program 1 (time)
- More efficient (space)
- More clarity (strategy)
- Easier to change (strategy)
- Cannot extend to three dimension

PROGRAM 2'

This program is identical to program 2 except for one change in the representation of the board. The board positions to vector elements as follows:

8	3	4
1	5	9
6	7	2

Notice that this numbering of the board produces a magic square: all the rows, columns, and diagonals sum up to 15. This means that we can simplify the process of checking for a possible win. In addition to marking the board as moves are made, we keep a list, for each player, of the squares in which he or she has played.

S = sum of two paired owned by a player D = 15 – S

if $0 < D < 10$ and Board [D] is empty then the player can win

If this difference is not positive or if it is greater than 9, then the original two squares were not collinear and so can be ignored. Otherwise if the square representing the difference is blank, a move there will produce a win. Since no player can have more than four squares at a time, there will be many fewer squares examined using this scheme than there were using the more straightforward approach of program 2.

Comments:

This comparison raises an interesting question about the relationship between the way people solve problems and the way computer do. Why do people find the row-scan approach easier while the number-counting approach is more efficient for a computer? We do not know enough about how people work to answer that question completely.

PROGRAM 3

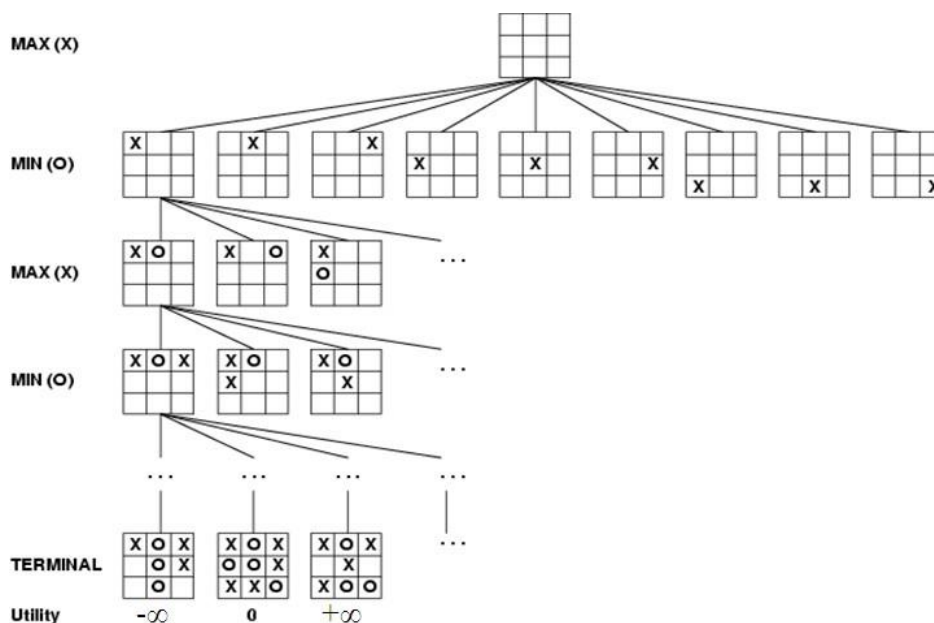
Data Structures

BoardPosition: A structure containing a nine-element vector representing the board, a list board positions that could result from the next move, and a number representing an estimate of how likely the board position is to lead to an ultimate win for the player to move.

The Algorithm: To decide on the next move, look ahead at the positions that result from each possible move. Decide which position is best, make the move that leads to that position, and assign the rating of that best move to the current position.

To decide which of a set of board positions is best, do the following for each of them:

1. See if it is a win. If so, call it the best by giving it the highest possible rating.
2. Otherwise, consider all the moves the opponent could make next. See which of them is worst for us. Assume the opponent will make that move. Whatever rating those move has, assign it to node we are considering.
3. The best node is then the one with the highest rating.



Comments

- This program will require much more time than either of the others since it must search a tree representing all possible move sequences before making each move.
- Extendable.